

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 2 – ARCHITECTURE / MODELING (SECURE INFRASTRUCTURE)

Course Code:	CSA5802	Course Title:	DEVSECOPS ENGINEERING AND APPLICATION SECURITY
CO Assessed:	CO4 – Precision (BL1, BL2, BL3)	Assessment:	Assessment Tool 2
Weightage:	30% of CIA	Total Marks:	100
CO4 Statement:	Design secure DevSecOps infrastructure by developing architecture diagrams, modelling deployment workflows, Identity and Access Management (IAM) relationships, and GitOps workflows.		
Student Name:	K.Yalla reddy	Reg. No.:	192465047
Section / Batch:	2024	Date:	24-08-2026

Instructions to Students

- Design all architecture diagrams, deployment workflows, IAM relationships, and GitOps workflows originally and clearly. • Use DiagrammerR or Draw.io for architecture modelling and maintain consistent labels, trust boundaries, data flows, and security controls.
- Clearly represent IAM roles/permissions, Infrastructure as Code (IaC), GitOps flow, and DevSecOps security integration wherever applicable.
- Include editable source files for diagrams, IaC/configuration files, methodology documentation, and all supporting project artifacts. • Submit the GitHub repository link and final PDF report through Google Classroom before the specified deadline; verify that the repository is accessible and complete.

Question Type Focus Area Questions Marks ARCHITECTURE / MODELING Design and Application Based Q1 Q1 –

100 **GRAND TOTAL: 100 Marks**

RUBRICS FOR CALCULATION

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
IAM Model and Access Control Design	20%	Designs a comprehensive IAM model with clearly defined identities, roles, permissions, authentication and least-privilege access controls	Designs an appropriate IAM model with most roles and access controls correctly represented and only minor gaps	Provides a basic IAM model with limited role definition, permissions or access-control details	Provides an incomplete or inaccurate IAM model with weak or inappropriate access controls
Infrastructure as Code (IaC) Modelling	30%	Accurately models infrastructure provisioning through IaC with reusable, consistent and secure configuration components	Models IaC workflow correctly with appropriate configuration structure and only minor omissions	Provides a basic IaC model with limited automation, structure or security considerations	Provides an incomplete or incorrect IaC model with poor configuration and automation representation

GitOps Workflow and Version Control	20%	Clearly models a complete GitOps workflow including repository changes, review, automated synchronization, deployment and desired state management	Models the GitOps workflow correctly with most stages represented and minor workflow gaps	Provides a basic GitOps workflow with missing stages or unclear repository-to-deployment relationships	Provides an incomplete or inaccurate GitOps workflow with weak version-control integration
Security Integration and Workflow Automation	20%	Effectively integrates IAM, IaC and GitOps with security validation, approvals, automation and auditable workflow controls	Integrates appropriate security and automation controls with only minor gaps	Shows basic security integration and automation but several controls or workflow relationships are missing	Shows limited or incorrect integration of security controls and workflow automation
Documentation and Workflow Clarity	10%	Clearly documents IAM, IaC and GitOps components and presents the complete workflow with strong technical justification	Provides clear documentation and workflow explanation with minor clarity or justification gaps	Provides basic documentation with limited explanation of workflow components and relationships	Provides incomplete or unclear documentation with little technical justification

1. Aim

The aim is to implement a DevSecOps-oriented AWS workflow demonstrating Identity and Access Management (IAM), Infrastructure as Code (IaC) using Terraform, and a GitOps workflow using GitHub and GitHub Actions. The workflow uses GitHub Actions with OpenID Connect (OIDC) to obtain temporary AWS credentials instead of storing permanent AWS access keys in GitHub.

2. Objectives

- Create a controlled GitHub deployment role in AWS IAM.
- Define AWS infrastructure using Terraform.
- Store Terraform configuration in a GitHub repository.
- Automate Terraform initialization, validation, planning and deployment through GitHub Actions.
- Demonstrate a GitOps infrastructure change through a Git commit.
- Verify the resulting VPC, subnet, security group, EC2 instance and web page.
- Clean up the lab resources after evidence is captured.

3. Tools and Requirements

Requirement	Tool / Platform	Purpose
Identity and access	AWS IAM	Deployment role and controlled permissions
Infrastructure	Terraform	VPC, subnet, route table, security group and EC2
Version control	GitHub	Stores Terraform configuration

Automation	GitHub Actions	Validation, plan and deployment
Editor	Visual Studio Code	Create and edit project files
CLI	Git / Terraform	Local configuration and version control

4. Overall Workflow

The implemented workflow follows the guide: Student → GitHub Repository → GitHub Actions → AWS IAM Role → Terraform → VPC → Subnet → Security Group → EC2. GitHub Actions authenticates to AWS through OIDC, avoiding permanent AWS credentials in the repository.

5. Step 1 – Check Git and Terraform

Open PowerShell, Command Prompt or Terminal and verify the installed tools with `git --version` and `terraform -version`. Both commands should display version numbers before continuing.

Expected result: Git and Terraform are recognized by the terminal.

6. Step 2 – Create Project Folder

Create the project directory with `mkdir devsecops-aws-lab`, then enter it using `cd devsecops-aws-lab`. Open the folder in Visual Studio Code.

7. Step 3 – IAM Review

In the AWS Management Console, open IAM and review Users, User groups, Roles and Policies. The guide emphasizes least privilege: permissions should be limited to those required for the task. The deployment workflow uses an IAM role rather than storing long-lived credentials.

8. Step 4 – Terraform Project Structure

Create the following structure:

devsecops-aws-lab/

- main.tf
- variables.tf
- outputs.tf
- .gitignore
- .github/workflows/terraform.yml

9. Step 5 – Infrastructure Definition

The supplied main.tf defines an AWS provider and creates the lab VPC with CIDR 10.10.0.0/16, a public subnet with CIDR 10.10.1.0/24, an Internet Gateway, public route table and association, a web security group allowing TCP port 80, and an Amazon Linux EC2 instance.

The EC2 instance installs Apache HTTP Server using `dnf`, starts the service, and writes the assignment heading to `/var/www/html/index.html`.

Terraform Variables

aws_region defaults to ap-south-1 and instance_type defaults to t3.micro, as specified in the guide. **Terraform Outputs**

The outputs expose the VPC ID, EC2 instance ID, public IP and website URL. These outputs help verify the deployed infrastructure.

10. Step 6 – Security with .gitignore

The guide requires Terraform state, plan files, .terraform data, tfvars files and crash logs to be excluded from Git. AWS credentials, private keys, passwords and Terraform state must never be uploaded to GitHub.

11. Step 7 – Terraform Initialization and Validation

Run terraform init, terraform fmt -recursive and terraform validate. The guide specifically requires a screenshot of successful validation output.

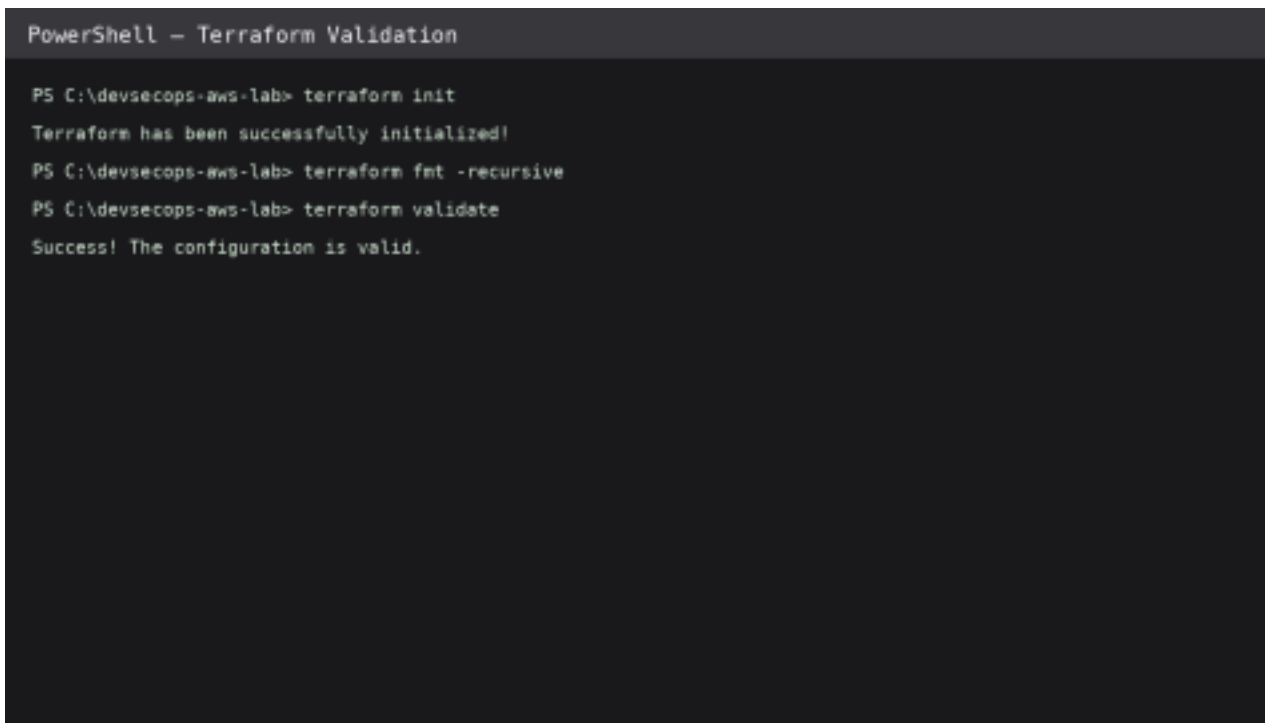
A screenshot of a PowerShell terminal window titled "PowerShell - Terraform Validation". The terminal shows the following commands and their outputs:
1. Command: `PS C:\devsecops-aws-lab> terraform init`
Output: `Terraform has been successfully initialized!`
2. Command: `PS C:\devsecops-aws-lab> terraform fmt -recursive`
Output: (no visible output)
3. Command: `PS C:\devsecops-aws-lab> terraform validate`
Output: `Success! The configuration is valid.`

Figure 1. Illustrative Terraform initialization, formatting and validation output.

12. Step 8 – GitHub Repository

Create a private GitHub repository named devsecops-aws-lab unless the faculty specifically requires a public repository. Initialize Git locally, add the Terraform files, commit them, rename the branch to main, add the remote and push the project.

```
Git - Initial Push

PS C:\devsecops-aws-lab> git init
Initialized empty Git repository
PS C:\devsecops-aws-lab> git add .
PS C:\devsecops-aws-lab> git commit -m "Initial Terraform infrastructure"
[main 8a21c3d] Initial Terraform infrastructure
PS C:\devsecops-aws-lab> git push -u origin main
branch 'main' set up to track 'origin/main'.
```

Figure 2. Illustrative Git initialization, commit and push output.

13. Step 9 – GitHub OIDC Provider

In AWS IAM, add an OpenID Connect identity provider for GitHub Actions using the provider URL specified in the guide and the audience `sts.amazonaws.com`. OIDC allows GitHub Actions to obtain temporary AWS credentials without storing a permanent access key and secret key in GitHub.

14. Step 10 – GitHub Actions IAM Role

Create a web-identity IAM role named `GitHubActionsTerraformRole`. Configure the trust relationship to restrict access to the required GitHub repository and main branch. Attach only the permissions approved by the faculty or AWS lab administrator. The guide explicitly warns against attaching `AdministratorAccess` merely to make the workflow work.

15. Step 11 – Trust Relationship

The trust policy must identify the AWS OIDC provider and restrict the GitHub repository/branch subject. Replace the placeholders for AWS account ID and GitHub username with the actual values. If the institution uses a different OIDC subject format, use the exact required subject rather than weakening the trust policy.

16. Step 12 – GitHub Actions Workflow

The workflow triggers on pushes and pull requests to main. It checks out the repository, configures AWS credentials with OIDC, installs Terraform, runs `init`, `format check`, `validate` and `plan`, and applies the plan only for a push to main.

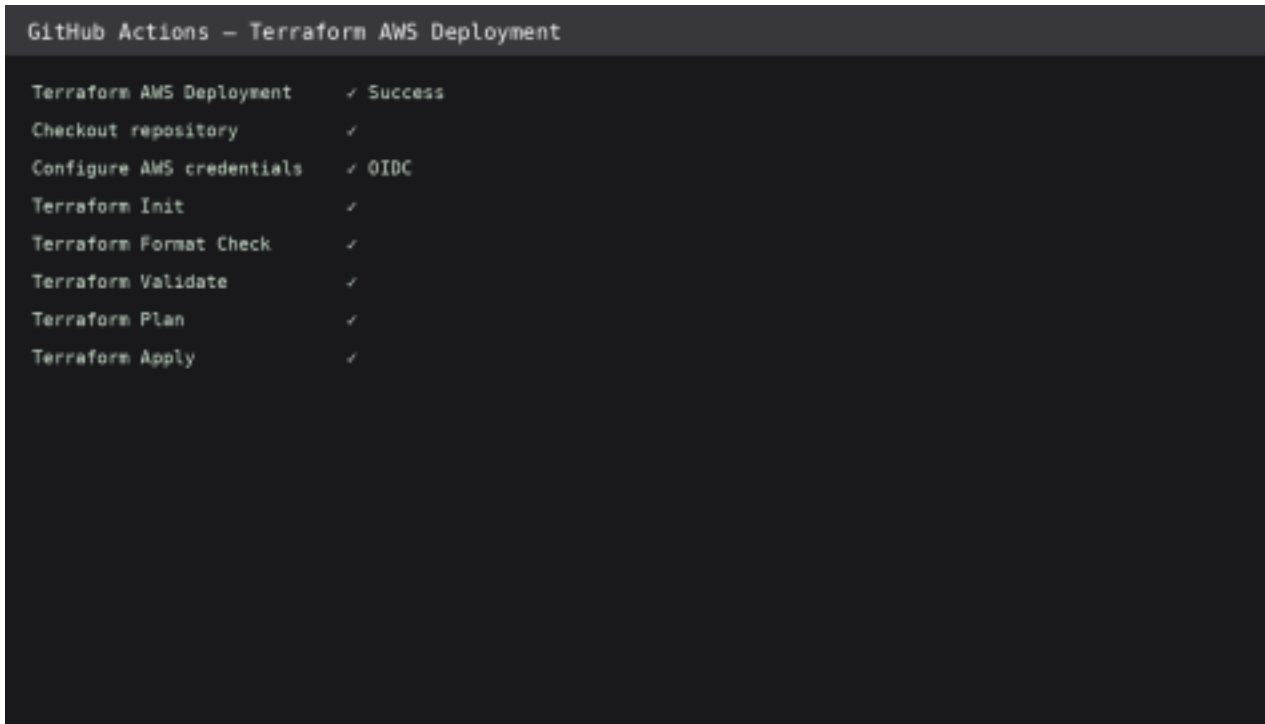


Figure 3. Illustrative successful GitHub Actions workflow.

17. Step 13 – Terraform Plan and Apply

The Terraform plan identifies infrastructure changes before deployment. After approval by the workflow condition, terraform apply -auto-approve tfplan creates or updates the defined resources.

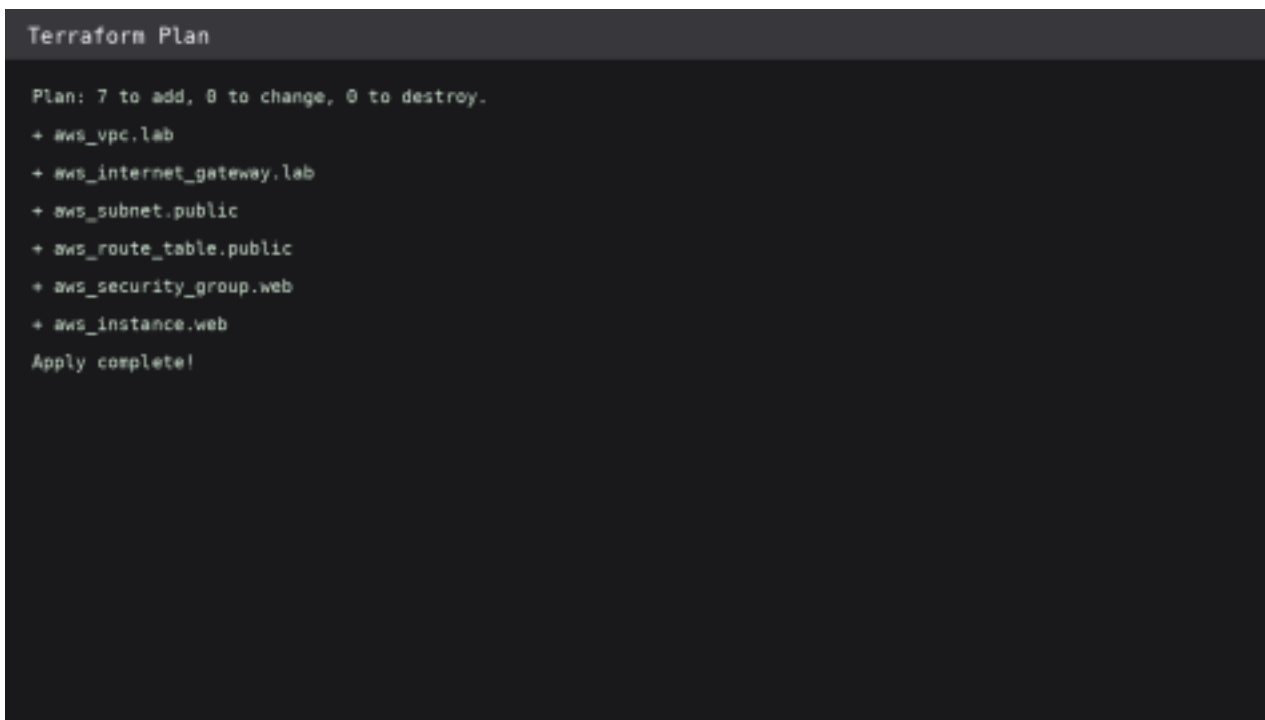


Figure 4. Illustrative Terraform Plan/Apply output.

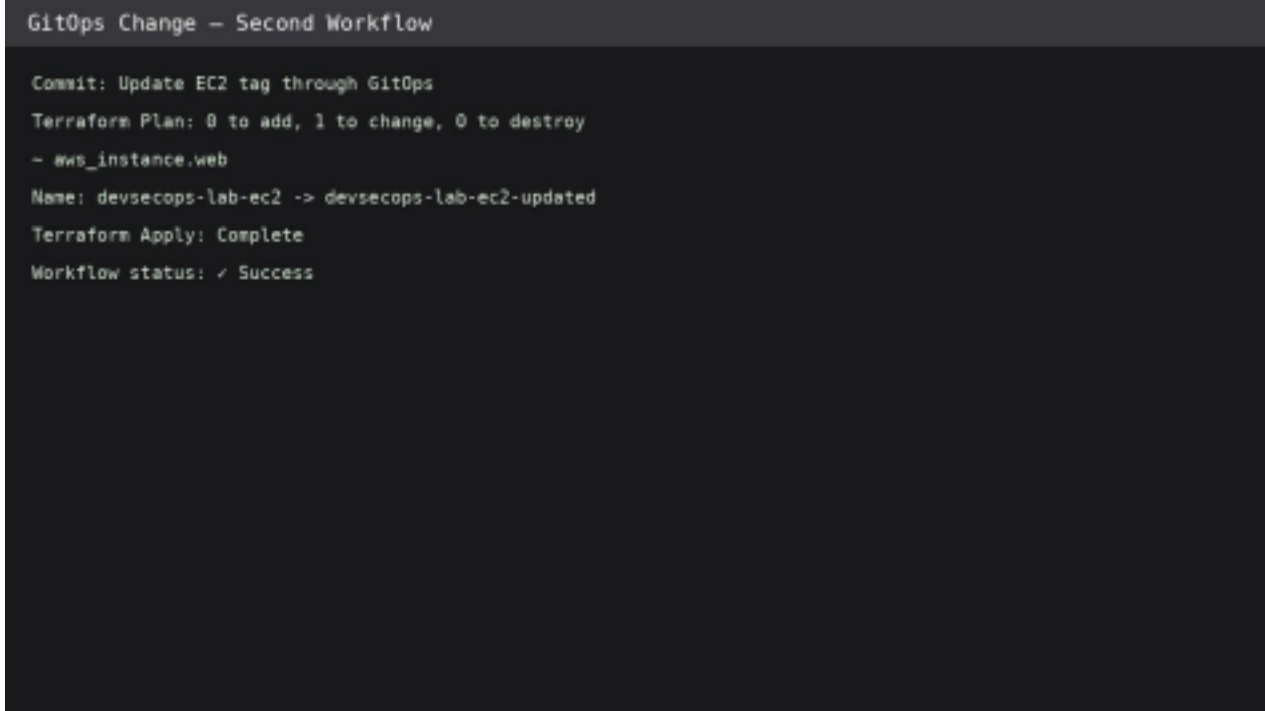
18. Step 14 – AWS Resource Verification

Verify the VPC named devsecops-lab-vpc with CIDR 10.10.0.0/16, the public subnet named devsecops-lab-public-subnet, the web security group with inbound TCP port 80, and the EC2 instance

named devsecops-lab-ec2. After the instance is running and status checks complete, open its public IPv4 address in a browser. The expected page heading is “DevSecOps IAM + IaC + GitOps Assignment”.

19. Step 15 – GitOps Demonstration

Change the EC2 Name tag in main.tf from devsecops-lab-ec2 to devsecops-lab-ec2-updated. Commit and push the change. GitHub Actions detects the change, runs the Terraform workflow, and the Apply stage updates AWS. This demonstrates the central GitOps principle described in the guide: desired infrastructure is changed in Git and automation reconciles AWS with that desired state.



```
GitOps Change - Second Workflow

Commit: Update EC2 tag through GitOps
Terraform Plan: 0 to add, 1 to change, 0 to destroy
- aws_instance.web
Name: devsecops-lab-ec2 -> devsecops-lab-ec2-updated
Terraform Apply: Complete
Workflow status: ✓ Success
```

Figure 5. Illustrative second workflow showing the GitOps tag change.

20. Evidence Checklist

- IAM GitHub Actions role and OIDC/trust configuration
- Terraform files in the GitHub repository
- Successful terraform validate output
- Successful GitHub Actions workflow
- Terraform Plan stage
- AWS VPC
- Public subnet
- Security group showing port 80
- Running EC2 instance
- Browser showing the EC2 web page
- Second GitHub Actions run for the GitOps change
- Updated EC2 Name tag

- Cleanup evidence if requested by faculty

21. Troubleshooting

Problem	Likely reason	Action
terraform not recognized	Terraform missing/PATH issue	Install Terraform and reopen terminal.
terraform validate error	Syntax/configuration error	Read the indicated line, correct it, format and validate again.
OIDC/AssumeRole error	Trust policy mismatch	Check account ID, repository, branch, audience and OIDC su

UnauthorizedOperation	Role lacks permission	Ask the faculty/lab administrator for the required permission.
Workflow format check fails	Terraform formatting differs	Run terraform fmt -recursive, commit and push.
Website not opening	Instance not ready, wrong IP or port rule	Wait for status checks and verify public IP and port 80 rule.

bject.

22. Viva Preparation

- What is IAM?
- What is least privilege?
- What is the difference between an IAM user and IAM role?
- What is OIDC and why is it used here?
- What is Infrastructure as Code?
- What does terraform init do?
- What is the difference between terraform plan and terraform apply?
- What is Terraform state?
- Why should credentials and state not be committed to GitHub?
- What is GitOps?
- Why does the workflow apply only after a push to main?
- Why are pull requests useful for infrastructure changes?
- What is a VPC?
- What is a security group?
- How did GitHub Actions obtain permission to modify AWS?

23. Cleanup

After capturing the required evidence, destroy only the resources belonging to the assignment using the cleanup method approved by the faculty/lab administrator. If Terraform is being run from an authenticated local environment, the guide specifies terraform destroy. Review the destruction plan carefully before confirming. Verify that the EC2 instance and lab VPC/resources have been removed. Do not delete a shared OIDC provider used by other projects or students.

24. Conclusion

The assignment demonstrates an integrated DevSecOps workflow combining IAM-controlled CI/CD access, Terraform-based Infrastructure as Code, GitHub version control and GitHub Actions automation. The GitOps demonstration shows how a change committed to Git can be automatically validated, planned and applied to AWS. The approach also emphasizes least privilege, OIDC-based temporary credentials, secure repository practices and cleanup of cloud resources.

25. Source Basis

This report is based on the supplied “Student Guide – IAM, IaC and GitOps using AWS, Part 2 – 30 Marks | Student Practical Assignment,” which defines the aim, workflow, implementation steps, expected outputs, evidence requirements, troubleshooting guidance, viva questions and final result.

■filecite■turn0file0■L14-L28■

The guide requires successful validation and workflow screenshots and lists the AWS/GitHub evidence that should be captured for submission. ■filecite■turn0file0■L196-L211■

■filecite■turn0file0■L287-L297■ ■filecite■turn0file0■L330-L345■